

Reporte Técnico de Actividades Práctico-Experimentales Nro. 001

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Jimmy Ontaneda Fernando Patiño Ana Panamito Daniel Saavedra Francis Valdiviezo
Asignatura	Sistemas Digitales
Ciclo	5 A
Unidad	1
Resultado de aprendizaje de la unidad	R1. Diseña arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	001
Tipo	Grupal
Título de la Práctica	Uso de Wokwi y Microchip Studio en microcontroladores AVR: arquitectura, puertos de E/S y operaciones a nivel de bit
Nombre del Docente	Luis Darío Sinche C
Fecha	Viernes, 2 de Octubre del 2026
Horario	11h30 – 13h30
Lugar	Laboratorio
Tiempo planificado en el Sílabo	2 horas

2. Objetivo(s) de la Práctica

- Objetivo general. Reconocer la arquitectura del microcontrolador ATmega328P y controlar sus periféricos de entradas/salidas digitales escribiendo y leyendo directamente sus registros mediante operaciones a nivel de bit, usando simulación en Wokwi y Microchip Studio. Objetivos específicos: Relacionar los pines de la placa Arduino UNO con los puertos y bits del ATmega328P (PB, PC, PD). Configurar entradas y salidas

digitales con los registros DDRx, PORTx y PINx usando máscaras (AND, OR, XOR, NOT y desplazamientos). Comparar el acceso mediante la API de Arduino con el acceso directo a registros, en tiempo de ejecución. Identificar en el código ensamblador generado las instrucciones lógicas y de manejo de bits del set de instrucciones AVR (SBI, CBI, SBIS, IN, OUT, ORI, EOR).

3. Materiales, Reactivos

- No se requieren componentes físicos: toda la práctica es simulada. Componentes virtuales usados en Wokwi: 1 Arduino UNO (ATmega328P a 16 MHz). 4 LEDs (rojo, amarillo, verde y azul) y 4 resistencias de 220 Ω . 1 pulsador.

4. Equipos y Herramientas

- Computador con navegador web y conexión a internet.
- Wokwi (<https://wokwi.com>), con cuenta gratuita para guardar y compartir los proyectos. FEIRNNR - Carrera de Computación Microchip Studio 7 (solo Windows), con el simulador del ATmega328P.
- Si no dispone de Windows, use Compiler Explorer (<https://godbolt.org>) para la Parte E.
- Hoja de datos del ATmega328P y AVR Instruction Set Manual (Microchip Technology). Cuenta de GitHub y acceso al EVA para la entrega.

5. Procedimiento / Metodología

Parte A. Reconocimiento del entorno y del microcontrolador

- Identificamos en la placa simulada el chip ATmega328P, el LED integrado "L" y el pin D13. La cual el puerto correspondiente es.. y el bit..
- Iniciamos la simulación con el código correspondiente, donde verificamos que le LED "L" si parpadea

Parte B. El mismo comportamiento, ahora con registros

- En el Código 2 verificamos y el comportamiento es idéntico al del Código 1.
- Verificamos que se enciendan y se apaguen el LED con un `delay(500)` entre ellas.
- Al momento de sustituir la conmutación, el ATmega328P permite, que los registros PINx realicen un toggle de los bits de PORTx al escribir un `1` en el bit correspondiente.
- una vez que nos funciona el `PORTB = (1 << PORTB5);` luego comprobamos que a los otros bits puede afectar a otras salidas porque sobrescribe todo el registro `PORTB`. Esto se comprobará en la Parte C al trabajar con varias salidas simultáneamente.

Parte C. Circuito y laboratorio de bits

Tabla 3. Conexiones pin a pin

Componente	Pin componente del	Pin Arduino UNO	Puerto / bit	Configuración
LED0 (rojo) + R0 220 Ω	Ánodo (A) vía R0	D8	PB0	Salida
LED1 (amarillo) + R1 220 Ω	Ánodo (A) vía R1	D9	PB1	Salida
LED2 (verde) + R2 220 Ω	Ánodo (A) vía R2	D10	PB2	Salida

LED3 (azul) + R3 220 Ω	Ánodo (A) vía R3	D11	PB3	Salida
LED0 ... LED3	Cátodo (C)	GND	—	—
Pulsador	1.l	D2	PD2	Entrada con pull-up interna
Pulsador	2.l	GND	—	—
LED integrado "L"	—	D13	PB5	Salida (latido)

- Al presionar el pulsador podemos observar los tres modos de funcionamiento. Cada modo genera un patrón diferente en los LEDs mediante una operación a nivel de bits.
- Al observar la línea **PORTB** = ... en el Monitor Serie, identifiqué que cada bit representa el estado de una salida del puerto.
- Al realizar la prueba observamos que el LED "L" también podía cambiar de estado, debido a que la asignación modifica todos los bits del registro **PORTB**, incluido el bit correspondiente a PB5.

Tabla 4. Configuración de registros

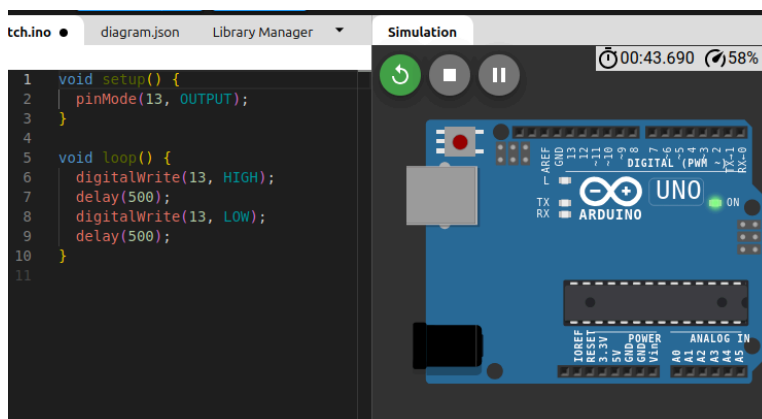
Pin	Función	Bit DDRx	Bit PORTx	Bit PINx leído
D8 (PB0)	LED0 encendido			—
D11 (PB3)	LED3 apagado			—
D2 (PD2)	Pulsador suelto			
D2 (PD2)	Pulsador presionado			
D13 (PB5)	LED "L" encendido			—

Parte D. Costo de la API frente al acceso a registros

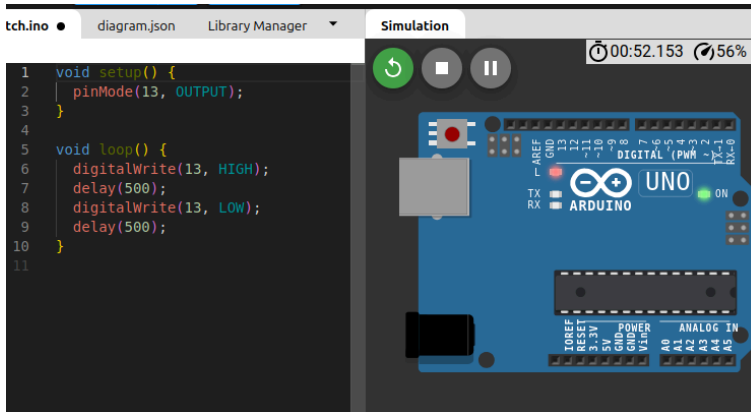
- Registramos los tiempos de las tres ejecuciones, calculamos el promedio y la relación. en los tiempos incluimos las sobrecargas del bucle y de la interrupción del Timer0

6. Resultados

A1.

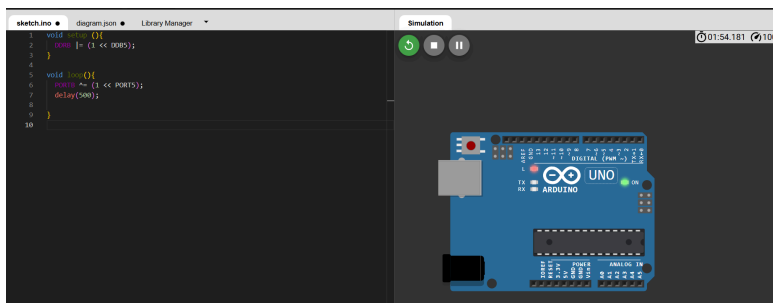


Se creó un nuevo proyecto en Wokwi utilizando una placa Arduino UNO y se guardó con el nombre correspondiente a la práctica.



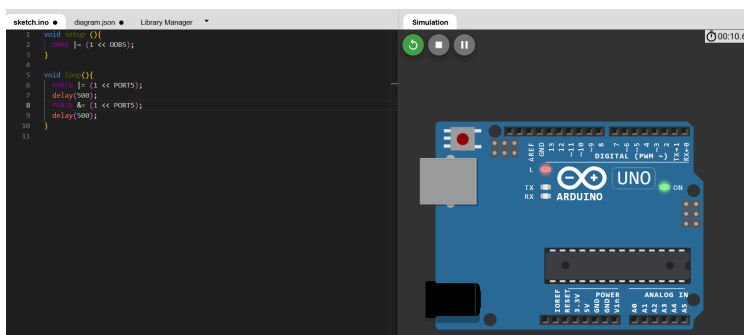
Se pudo identificar que el pin digital D13 del Arduino uno corresponde al puerto PB5 del microcontrolador ATmega328P. También se verificó que este pin está conectado al LED integrado llamado L de la placa.

B1.



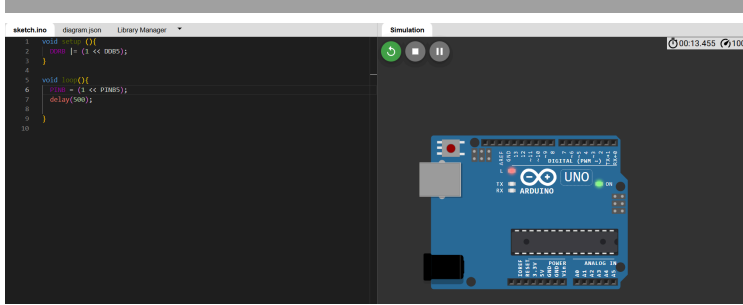
Se pudo verificar que al utilizar directamente los registros DDRB y PORTB, el LED L de la placa, se enciende y se apaga cada 500 ms de la misma manera que con digitalWrite(). Esto demuestra que se puede controlar el pin D13 directamente mediante los registros del microcontrolador.

B2.



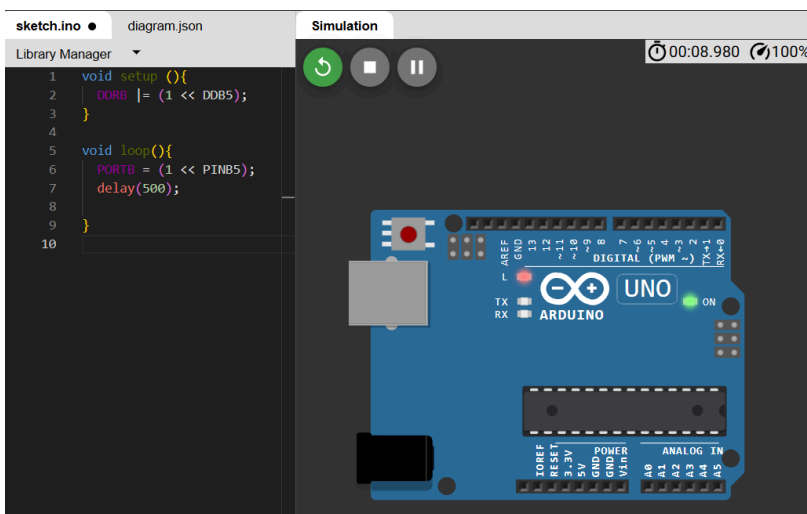
Se pudo verificar que al utilizar el código `PORTB |= (1 << PORTB5)` el led L se enciende, y al utilizar `PORTB &= ~(1 << PORTB5)` el LED se apaga. Con los retardos de 500 ms, el led continúa parpadeando de forma repetitiva.

B3.



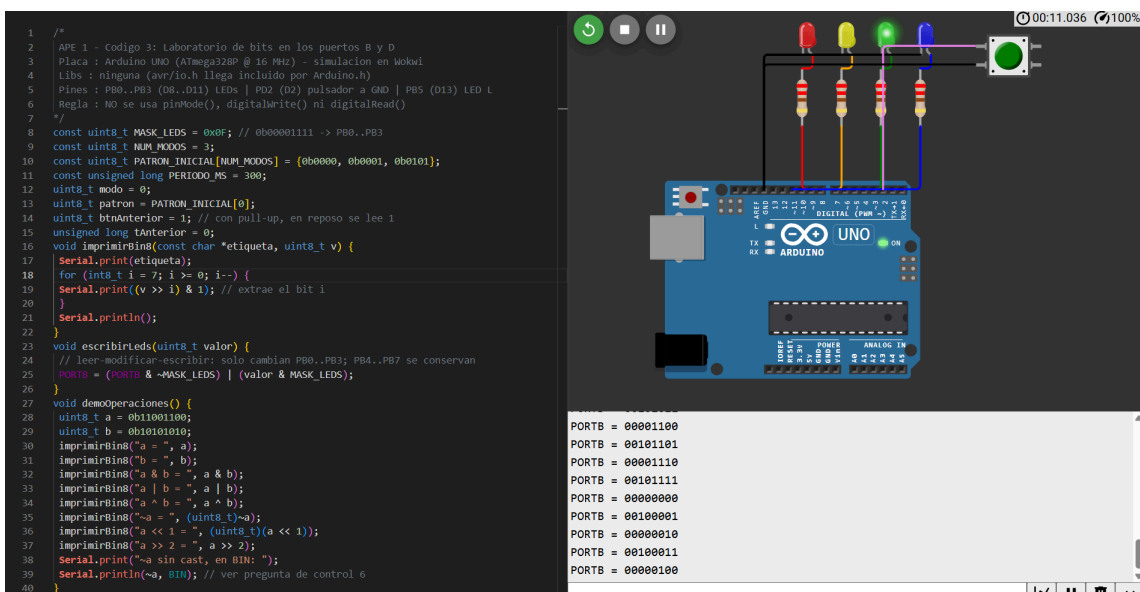
Se pudo verificar que al utilizar $PINB = (1 \ll PINB5)$, el led L cambia de estado cada 500 ms, es decir, si está apagado se enciende y si está encendido se apaga. Esto es posible porque el ATmega328P permite conmutar un bit de PORTB escribiendo un 1 en el bit correspondiente de PINB

B4.



Se pudo verificar que al poner el código de $PORTB = (1 \ll PINB5)$, el led L que corresponde al pin 13, se queda prendido todo el rato, sin prenderse y apagarse.

C1, C2



C3

Resultados de demoOperaciones()

a & b

a: 11001100 AND b: 10101010 = 10001000

a | b

a: 11001100 OR b: 10101010 = 11101110

a ^ b

a: 11001100 XOR b: 10101010 = 01100110

~a (invierte cada bit)

a: 11001100 -> ~a: 00110011

a << 1 (Desplaza los bits 1 posición a la izquierda e ingresa un 0 en el LSB)

11001100 << 1 = 11001100₂ -----> 10011000

a >> 2 (Desplaza los bits 2 posiciones a la derecha e ingresan ceros por la izquierda)

11001100 >> 2 = 00110011

```
b = 10101010
a & b = 10001000
a | b = 11101110
a ^ b = 01100110
~a = 00110011
a << 1 = 10011000
a >> 2 = 00110011
```

C4

Describe los tres modos. Para cada modo, explique qué operación a nivel de bit produce el patrón.

Modo 0

En este modo el patrón reinicia en PORTB = 00000000, los 4 LEDs incrementan en binario desde PORTB = 00000000 (0) hasta PORTB = 00101111 (15). Al llegar a 15, reinician a 0 de forma cíclica.

Operación a nivel de bit: `patron = (patron + 1) & MASK_LEDS;`

Se realiza una suma (+ 1) sobre la variable. La operación AND bit a bit (& MASK_LEDS, donde MASK_LEDS = 0b00001111) funciona como un filtro que elimina el acarreo que pase al bit 4, forzando a que la variable se mantenga estrictamente en el rango de 0 a 15.

Modo 1

Inicia en PORTB = 00100010. En cada ciclo, el LED encendido se desplaza una posición hacia la izquierda (LED0 → LED1 → LED2 → LED3). Cuando llega al LED3, el LED encendido "da la vuelta" e ingresa nuevamente por la derecha (LED0).

Operación a nivel de bit: `patron = ((patron << 1) | (patron >> 3)) & MASK_LEDS;`

1. `patron << 1`: Desplaza los bits una posición a la izquierda. El bit ubicado en la posición 3 se mueve a la posición 4.
2. `patron >> 3`: Toma el bit 3 y lo traslada a la posición 0.
3. `"|"` (OR bit a bit): Combina ambos desplazamientos para que el bit extraído de la izquierda reingrese por la derecha.
4. `"&"MASK_LEDS`: Conserva únicamente los 4 bits inferiores correspondientes a los pines PB0..PB3.

Modo 2

Inicia con el patrón `PORTB = 00101010` (LED0 y LED2 encendidos; LED1 y LED3 apagados). Cada 300ms, el estado de los 4 LEDs se invierte simultáneamente (`0101 ↔ 1010`), haciendo conmutar de forma alternada los LEDs pares e impares.

Operación a nivel de bits: `patron ^= MASK_LEDS`;

Los bits de los 4 LEDs cambian de 0 a 1 o de 1 a 0 en cada ciclo, mientras que los bits superiores (PB4..PB7) no se ven afectados al evaluarse contra ceros.

C5

Bits PB0 a PB3 corresponden a los LEDs externos (LED0, LED1, LED2 y LED3). Bit PB5 (Bit 5) corresponde al LED integrado "L" (Pin D13 del Arduino).

C6

El LED L deja de parpadear normal. Según la parte B4 no se debe usar asignación directa (`=`) sobre un registro de puerto cuando existen múltiples salidas independientes compartiendo dicho puerto, ya que no preserva la información de los pines vecinos.

C7

Configuración de registros

Pin	Funcion	Bit DDRx	Bit PORTx	Bit PINx leído
D8 (PB0)	LED0 encendido	1	1	-
D11 (PB3)	LED3 apagado	1	0	-
D2 (PD2)	Pulsador suelto	0	1	1
D2 (PD2)	Pulsador presionado	0	1	0
D13 (PB5)	LED "L" encendido	1	1	-

D1, D2.

sketch.ino

diagram.json

Library Manager

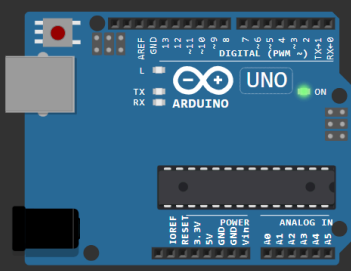
```

1 // APE 1 - Codigo 4: costo d
2 // Placa: Arduino UNO (ATmeg
3
4 const uint16_t N = 10000;
5
6 void setup() {
7   Serial.begin(9600);
8   pinMode(13, OUTPUT);
9
10  unsigned long t0 = micros(
11
12  for (uint16_t i = 0; i < N
13    digitalWrite(13, HIGH);
14    digitalWrite(13, LOW);
15  }
16
17  unsigned long t1 = micros(
18
19  for (uint16_t i = 0; i < N
20    PORTB |= (1 << PORTB5);
21    PORTB &= ~(1 << PORTB5);
22  }
23
24  unsigned long t2 = micros(
25
26  Serial.print(F("digitalWri
27  Serial.print((t1 - t0) / (
28  Serial.println(F(" us por
29
30  Serial.print(F("Registro
31  Serial.print((t2 - t1) / (
32  Serial.println(F(" us por

```

Simulation

00:22.242 100%



```

digitalWrite : 3.395 us por escritura
Registro      : 0.220 us por escritura

```

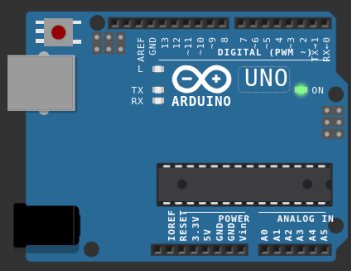
```

3
4 const uint16_t N = 10000;
5
6 void setup() {
7   Serial.begin(9600);
8   pinMode(13, OUTPUT);
9
10  unsigned long t0 = micros();
11
12  for (uint16_t i = 0; i < N; i++) {
13    digitalWrite(13, HIGH);
14    digitalWrite(13, LOW);
15  }
16
17  unsigned long t1 = micros();
18
19  for (uint16_t i = 0; i < N; i++) {
20    PORTB |= (1 << PORTB5);
21    PORTB &= ~(1 << PORTB5);
22  }
23
24  unsigned long t2 = micros();
25
26  Serial.print(F("digitalWrite : "));
27  Serial.print((t1 - t0) / (2.0 * N), 3);
28  Serial.println(F(" us por escritura"));
29
30  Serial.print(F("Registro      : "));
31  Serial.print((t2 - t1) / (2.0 * N), 3);
32  Serial.println(F(" us por escritura"));
33
34
35 void loop() {
36

```

Simulation

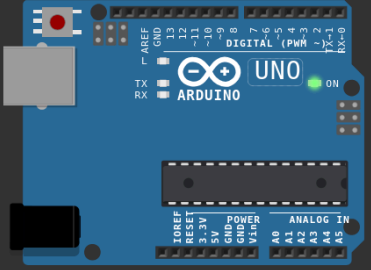
digitalWrite : 3.395 us por escritura
Registro : 0.220 us por escritura



```

3
4  const uint16_t N = 10000;
5
6  void setup() {
7      Serial.begin(9600);
8      pinMode(13, OUTPUT);
9
10     unsigned long t0 = micros();
11
12     for (uint16_t i = 0; i < N; i++) {
13         digitalWrite(13, HIGH);
14         digitalWrite(13, LOW);
15     }
16
17     unsigned long t1 = micros();
18
19     for (uint16_t i = 0; i < N; i++) {
20         PORTB |= (1 << PORTB5);
21         PORTB &= ~(1 << PORTB5);
22     }
23
24     unsigned long t2 = micros();
25
26     Serial.print(F("digitalWrite : "));
27     Serial.print((t1 - t0) / (2.0 * N), 3);
28     Serial.println(F(" us por escritura"));
29
30     Serial.print(F("Registro      : "));
31     Serial.print((t2 - t1) / (2.0 * N), 3);
32     Serial.println(F(" us por escritura"));
33 }
34
35 void loop() {
36 }

```

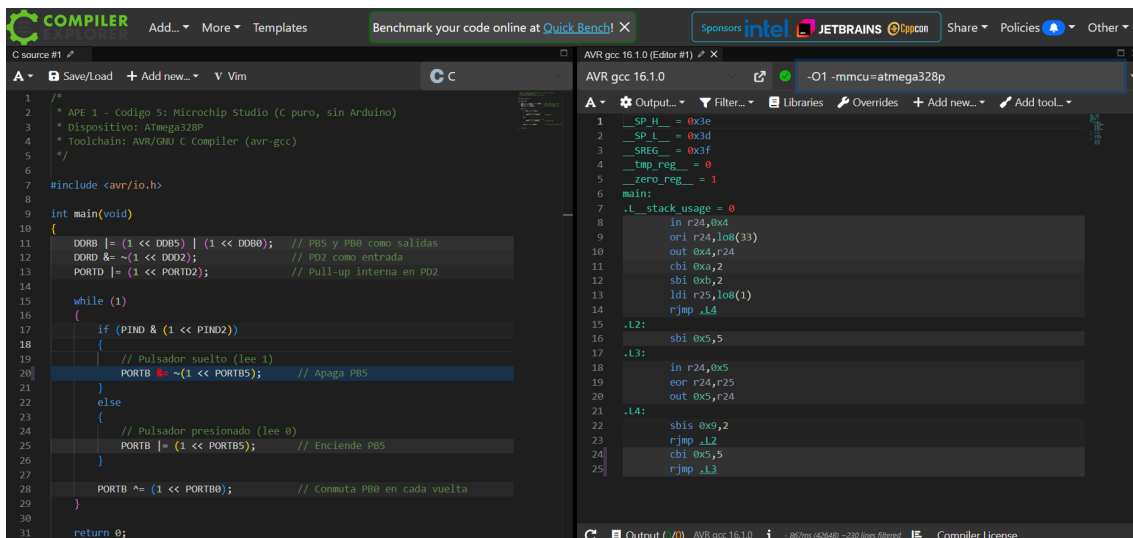


```

digitalWrite : 3.395 us por escritura
Registro      : 0.220 us por escritura

```

E1, E2:



```

1  /*
2   * APE 1 - Código S: Microchip Studio (C puro, sin Arduino)
3   * Dispositivo: ATmega328P
4   * Toolchain: AVR/GNU C Compiler (avr-gcc)
5   */
6
7  #include <avr/io.h>
8
9  int main(void)
10 {
11     DDRB |= (1 << DDB5) | (1 << DDB0); // PB5 y PB0 como salidas
12     DDOD &= ~(1 << DDD2); // PD2 como entrada
13     PORTD |= (1 << PORTD2); // Pull-up interna en PD2
14
15     while (1)
16     {
17         if (PIND & (1 << PIND2))
18         {
19             // Pulsador suelto (lee 1)
20             PORTB &= ~(1 << PORTB5); // Apaga PB5
21         }
22         else
23         {
24             // Pulsador presionado (lee 0)
25             PORTB |= (1 << PORTB5); // Enciende PB5
26         }
27
28         PORTB ^= (1 << PORTB0); // Conmuta PB0 en cada vuelta
29     }
30
31     return 0;

```

```

1  __SP_H__ = 0x3e
2  __SP_L__ = 0x3d
3  __SREG__ = 0x3f
4  __tmp_reg__ = 0
5  __zero_reg__ = 1
6  main:
7  .L_stack_usage = 0
8  in r24,0x4
9  ori r24,lo8(33)
10 out 0x4,r24
11 cbi 0x5,2
12 sbi 0x5,2
13 ldi r25,lo8(1)
14 rjmp .L4
15 .L2:
16 sbi 0x5,5
17 .L3:
18 in r24,0x5
19 eor r24,r25
20 out 0x5,r24
21 .L4:
22 sbis 0x0,2
23 rjmp .L2
24 cbi 0x5,5
25 rjmp .L3

```

Se realizó los puntos 1 y 2 de la sección E, trabajando desde Compiler Explorer

Línea en C	Instrucción(es) AVR	Ciclos
DDRB = (1 << DDB5) (1 << DDB0);	<pre> in r24,0x4 ori r24,lo8(33) out 0x4,r24 </pre>	3

DDRD &= ~(1 << DDD2);	<pre>cbi 0xa,2</pre>	2
PORTD = (1 << PORTD2);	<pre>sbi 0xb,2</pre>	2
if (PIND & (1 << PIND2))	<pre>sbis 0x9,2 rjmp .L2</pre>	2 si PIND2=1 / 3 si PIND2=0
PORTB &= ~(1 << PORTB5);	<pre>cbi 0x5,5 rjmp .L3</pre>	2
PORTB = (1 << PORTB5);	<pre>sbi 0x5,5</pre>	2
PORTB ^= (1 << PORTB0);	<pre>.L__stack_usage = 0 ldi r25,lo8(1) rjmp .L4 .L3: in r24,0x5 eor r24,r25 out 0x5,r24</pre>	3

E4.

Debido a que en la practica ocupamos Compiler Explorer en vez de Microchip Studio, no se pudo realizar la ejecución de la simulación. Se compilo el código 5 utilizando AVR GCC 16.0.1, con las opciones requeridas por el documento de la practica que se nos fue proporcionado "01 -mmcu=atmega328p" y analizamos el codigo que nos genero en ensamblador.

E5.

En la inicialización, los bits PB5 y PB0 del registro DDRB se configuran como salidas. El bit PD2 de DDRD se configura como entrada y el bit PD2 de PORTD se coloca en 1 para activar la resistencia pull-up interna.

Dentro del ciclo while, el programa consulta el bit PIND2. Cuando PIND2 = 1, el pulsador está suelto y el bit PB5 de PORTB se pone en 0, apagando la salida. Cuando PIND2 = 0, el pulsador está presionado y PB5 se pone en 1, encendiendo la salida. Además, PB0 cambia de estado en cada iteración mediante la operación XOR "PORTB ^= (1<< PORTB0);".

E6.

Para comenzar con el punto 6, comenzamos con el pulsador suelto "PIND2 = 1, lo cual no da como resultado

SBIS = 2 ciclos
CBI = 2 ciclos
RJMP = 2 ciclos
IN = 1 ciclo
EOR = 1 ciclo

OUT = 1 ciclo

En total 9 ciclos con -> "PIND2 = 1"

Con el pulsador presionado "PIND2 = 0"

Nos da como resultado

SBIS = 1 ciclo

RJMP = 2 ciclos

SBI = 2 ciclos

IN = 1 ciclo

EOR = 1 ciclo

OUT = 1 ciclo

En total 8 ciclos con -> 8 ciclos

Resumen realizado:

Suelto = 9 ciclos

Presionado = 8 ciclos

Links de Acceso

Literal A = <https://wokwi.com/projects/477137451715134465>

Literal B = <https://wokwi.com/projects/476789486182834177>

Literal C = <https://wokwi.com/projects/476791274788777985>

Literal D = <https://wokwi.com/projects/477137607225293825>

7. Preguntas de Control

1. **¿Qué diferencia a un microcontrolador de un microprocesador? Enumere los bloques del ATmega328P que identificó en la práctica.**

Un microcontrolador integra en un solo circuito el procesador, memoria y periféricos de entrada/salida, mientras que un microprocesador se centra principalmente en la unidad de procesamiento y necesita componentes externos para memoria y periféricos.

En el ATmega328P se identifican los bloques principales:

1. CPU o núcleo AVR.
2. Memoria Flash para el programa.
3. Memoria SRAM.
4. Memoria EEPROM.
5. Puertos digitales de entrada/salida: PORTB, PORTC y PORTD.
6. Temporizadores/contadores.
7. Convertidor analógico-digital (ADC).
8. Interfaz USART.
9. Interfaz SPI.
10. Interfaz I²C/TWI.

11. Interrupciones.
12. Reloj/oscilador.

2. **¿Qué implica que el ATmega328P tenga arquitectura Harvard? Relacione su respuesta con el uso de la macro F("...") en el Código 3.**

La arquitectura Harvard del ATmega328P posee buses y memorias independientes para el programa (Flash) y los datos (SRAM). Por defecto, los textos del código se copian a la limitada memoria SRAM (2 KB) al iniciar. La macro F("...") obliga al sistema a mantener y leer los textos directamente desde la memoria Flash (32 KB), evitando que se agote la SRAM.

3. **Explique por qué $PORTB = (1 \ll PORTB5)$; puede alterar otras salidas y $PORTB$**

$|= (1 \ll PORTB5)$; no, usando lo observado en los pasos B4 y C6.

$PORTB = (1 \ll PORTB5)$; hace una asignación directa. Pone un 1 en el bit 5 (LED L) pero borra y pone en 0 todos los demás pines del puerto. En los pasos B4 y C6 se observa que esto apaga los LEDs conectados en los otros pines.

$PORTB |= (1 \ll PORTB5)$; realiza una operación OR. Cambia a 1 el bit 5 sin alterar ni apagar el estado previo de los demás pines del puerto

4. **Para usar PD2 como entrada con pull-up, ¿qué valores deben tener DDRD2 y PORTD2? ¿Qué se lee en PIND2 con el pulsador suelto y presionado, y por qué?**

Registros: DDRD2 debe ser 0 (configura entrada) y PORTD2 debe ser 1 (activa la resistencia pull-up interna).

Lectura en PIND2: Con el pulsador suelto se lee 1 porque la resistencia interna mantiene el pin alimentado a VCC. Con el pulsador presionado se lee 0 porque el pin se conecta directamente a tierra (GND).

5. **Calcule a mano $0b11001100$ & $0b10101010$, $|$ y $\wedge$, y verifique sus resultados con el Monitor Serie.**

La "&" significa AND, se aplica la regla: $1 \text{ y } 1 = 1$; de lo contrario 0:

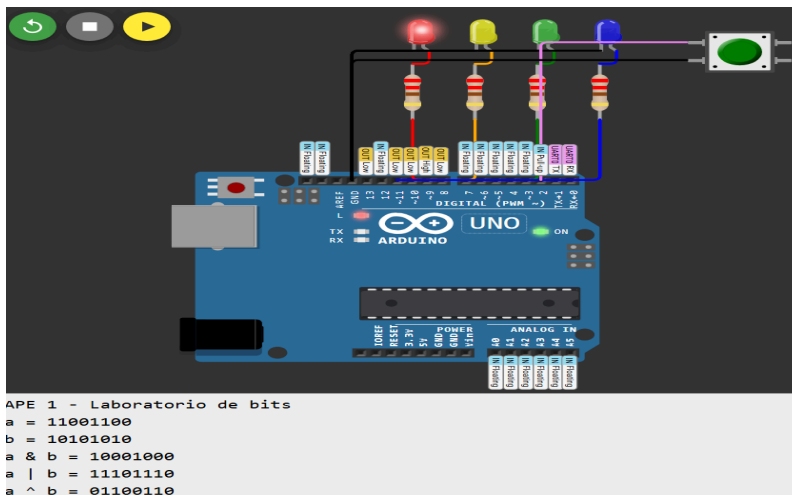
a: $0b11001100$ AND b: $0b10101010 = 10001000$

La "|" significa OR, se aplica la regla: $0 \text{ o } 0 = 0$; de lo contrario 1:

a: $0b11001100$ AND b: $0b10101010 = 11101110$

La " $\wedge$ " significa XOR, aplica la regla 1 si los bits son distintos; 0 si son iguales:

a: $0b11001100$ AND b: $0b10101010 = 01100110$



6. Si a es de tipo uint8_t, ¿por qué Serial.println(~a, BIN) imprime muchos más de 8 bits? ¿Cómo lo corrige el Código 3?

Al aplicar el operador NOT (~) a una variable de tipo uint8_t, ocurre una promoción de tipo entera (*integer promotion*). El valor se convierte automáticamente a un entero firmado de 16 o 32 bits (int) antes de invertir sus bits. Si el bit más significativo cambia a 1, la extensión de signo llena los bits superiores con unos, provocando que Serial.println() imprima 16 o más bits en formato binario.

Se corrige aplicando un moldeado explícito de tipo (*type casting*) a 8 bits: (uint8_t)~a. Esto recorta el resultado a un valor sin signo de 8 bits antes de pasarlo a la función de impresión.

7. Con los datos de la Parte D, explique por qué digitalWrite() es más lento que el acceso directo a registros. ¿Cuándo conviene cada enfoque (portabilidad frente a rendimiento)?

Por qué es más lento, digitalWrite() realiza múltiples validaciones y búsquedas en tiempo de ejecución: traduce el número de pin de Arduino al puerto y bit físico del ATmega3APE1_APE1_APE1_28P mediante tablas en memoria Flash, apaga la salida PWM si el pin la tiene activa, deshabilita interrupciones para operaciones seguras y valida los parámetros de entrada. El acceso directo a registros reemplaza todo este proceso por una sola instrucción en ensamblador a nivel de hardware.

8. ¿Qué instrucción AVR generó PORTB |= (1 << PORTB5); y cuántos ciclos toma?

Cuándo conviene cada enfoque:

- digitalWrite() conviene en prototipos rápidos o proyectos donde importa la portabilidad del código entre distintas placas (como Arduino UNO, ESP32 o Mega), ya que abstrae la arquitectura del chip.
- El acceso a registros conviene cuando se requiere máximo rendimiento, tiempos de respuesta precisos en microsegundos o cuando se ejecutan tareas intensivas dentro de interrupciones (ISR).

¿Por qué $PORTB \wedge= (1 \ll PORTB0)$; no se tradujo a una sola instrucción?

9. **Sostenibilidad: indique dos formas en que lo trabajado (simular antes de comprar o montar, acceso eficiente a los periféricos) contribuye a reducir el consumo energético o los residuos electrónicos.**

Simular antes de realizar la conexión a corriente previene el daño permanente en algunos casos por errores de cableado, sobrevoltaje o cortocircuitos, lo cual es algo notorio en la reducción de desechos en microcontroladores dañados y demás componentes que quedan obsoletos.

10. **Ventajas y limitaciones de simular en Wokwi frente al hardware real (rebote del pulsador, límites de corriente, tiempos).**

Woki y otros tipos de sistemas simuladores en línea evita realizar compras ayudándonos en nuestra economía en lo máximo posible para fomentar el aprendizaje, ayuda de forma directa al ambiente reduciendo los costos típicos como embalaje, transporte y temas de logística de entrega. En ciertos casos la experiencia de ensamblar estos circuitos es un factor sumamente importante para el aprendizaje de estudiantes así como el hecho que estamos en un entorno perfecto o ideal en el cual se ignoran demasiados factores que suceden en la realidad, como los límites de corriente y como se ven afectados por la temperatura del ambiente, la capacidad computacional real del microcontrolador

8. Conclusiones

- Al identificar actores y casos de uso, se concluye que un análisis en

9. Bibliografía

[1] W. P. Chamba Zaragocín, E. R. Guamán Quinche, F. J. Alvarez Pineda, J. O. Guamán Quinche, G. B. Zaragocín Arrobo y G. Á. Valarezo Benítez, Análisis y diseño preliminar de software basado en el modelado de requisitos y casos de uso. Un enfoque teórico y práctico. Loja, Ecuador: Universidad Nacional de Loja, 2025. Disponible en: <https://unl.edu.ec/investigacion/produccion-cientifica/analisis-y-diseno-preliminar-de-software-basado-en-el-modelado>